

Relatório do Trabalho Prático 1

Programação Paralela com OpenMP

Êxodo Melo, Thiago Aparecido

15 de Abril de 2026

Resumo

O presente relatório tem o objetivo apresentar o primeiro trabalho feito sobre a disciplina de Programação Paralela de paralelização da solução numérica de sistemas lineares utilizando o Método Iterativo de Jacobi utilizando a API OpenMP (Open Multi-Processing) e seus resultados.

1 Introdução

Sistemas aparecem em diversos locais, desde circuitos elétricos e eletrônicos à sistemas de controle industriais. É de interesse a busca de métodos para solucioná-los.

Um desses métodos é o Método de Jacobi para solução de sistemas lineares, caracterizada pela atualização independente de cada componente do vetor solução a partir dos valores da iteração anterior. A paralelização desse método surge do custo de $O(N^2)$ para cada iteração, tendo independência entre atualizações do vetor solução torna o método particularmente adequado para execução paralela, permitindo explorar arquiteturas de memória compartilhada de forma eficiente.

O método de Jacobi parte da seguinte lógica:

$$\begin{aligned} \sum_j A_{ij}x_j &= b_i \\ A_{ii}x_i + \sum_{j \neq i} A_{ij}x_j &= b_i \\ A_{ii}x_i &= b_i - \sum_{j \neq i} A_{ij}x_j \\ x_i &= \frac{b_i - \sum_{j \neq i} A_{ij}x_j}{A_{ii}} \end{aligned} \tag{1}$$

Tendo uma estimativa inicial para $\mathbf{x}$, pode-se atualizar iterativamente valendo pela equação. Para a convergência do método é precisa que a matriz $\mathbf{A}$ seja uma matriz de diagonal dominante, isto é:

$$|A_{ii}| > \sum_{j \neq i} |A_{ij}|, \forall i$$

A paralelização é um processo difícil e dependente de máquina, uma das maneiras de facilitar a implementação é a abstração de detalhes menos importantes para o programador. A Interface OpenMP oferece formas de implementação tal facilitam a paralisação.

Neste trabalho, é apresentada uma implementação do método de Jacobi utilizando a API OpenMP, com o objetivo de explorar o paralelismo de dados e avaliar o impacto da paralelização no desempenho do algoritmo. Além da implementação, são analisados aspectos como escolha do número de threads, política de escalonamento e comportamento de variáveis compartilhadas e privadas, bem como métricas de desempenho como tempo de execução, speedup e eficiência.

Dessa forma, o trabalho busca não apenas demonstrar a aplicação prática de técnicas de paralelização, mas também discutir as limitações e desafios associados à obtenção de ganho de desempenho com a solução paralela.

2 Implementação

O algoritmo implementado corresponde ao método iterativo de Jacobi para resolução de sistemas lineares, no qual o vetor solução é atualizado iterativamente a partir dos valores da iteração anterior. A paralelização foi realizada utilizando a API OpenMP, explorando o paralelismo no nível de dados presente nas iterações independentes do método.

Foi adotada a política de escalonamento padrão `static` na diretiva `#pragma omp parallel for`, uma vez que cada iteração do laço principal apresenta custo computacional aproximadamente uniforme. Em cada iteração externa, para um índice i , é calculada a soma dos produtos dos elementos da linha i da matriz pelos valores correspondentes do vetor solução atual. Como todas as iterações possuem a mesma complexidade ($O(n)$), a distribuição estática das iterações entre as threads minimiza o overhead associado ao escalonamento dinâmico e contribui para um melhor balanceamento de carga.

A primeira região paralela é responsável pelo cálculo do novo vetor solução (X_{new}). Cada thread computa independentemente um subconjunto das posições do vetor, utilizando apenas leituras do vetor X (imutável durante essa fase) e da matriz A . Essa característica garante ausência de dependências entre iterações, tornando o laço naturalmente paralelizável.

Na segunda região paralela, é realizado o cálculo do erro global da iteração, definido como a norma Euclidiana da diferença entre os vetores X_{new} e X . Para isso, foi utilizada a cláusula `reduction(+ : error)`, que permite acumular de forma segura os erros parciais computados por cada thread, evitando

condições de corrida. Além disso, nesta etapa ocorre a atualização do vetor X com os novos valores, preparando-o para a próxima iteração.

As estruturas de dados principais — matriz A e vetores B , X e X_{new} — são compartilhadas entre as threads, pois representam o estado global do sistema e precisam ser acessadas concorrentemente. Por outro lado, variáveis de controle dos laços (i , j) e variáveis temporárias como `sum` são privadas por padrão, garantindo independência entre as iterações e evitando interferência entre threads.

O critério de parada do algoritmo é baseado em dois fatores: o número máximo de iterações e a convergência do método, medida pelo erro calculado. A execução é interrompida quando o erro é menor que um valor limite (ϵ) ou quando o número máximo de iterações é atingido.

O número de threads é passado por argumento na chamada do programa. Não é especificado um número inicialmente pois é necessário realizar teste para encontrar a melhor performance, visto que para um número de threads muito grande, o custo de troca entre execuções aumentará.

3 Ambiente Experimental

O primeiro ambiente experimental de execução foi: Windows 10, Intel Core i5-8400 2.80GHz 16 GB RAM. Tendo o processador 6 núcleos físicos e número total de threads 6. Foi desenvolvido em C++ com o compilador `g++` (GNU C++ Compiler), versão 15.2.0 (MSYS2) e com a flag `-fopenmp`.

Foram geradas sistemas lineares convergentes de valores de A e x aleatórios de ordem 1000. Os testes foram executados variando o número de threads, com o objetivo de avaliar o impacto da paralelização no desempenho do método de Jacobi com valor de ϵ fixo sendo $\epsilon = 0.0001$.

Para cada configuração de número de threads, foram realizadas três execuções independentes, a fim de reduzir a influência de variações do sistema, como processos concorrentes e flutuações de desempenho. Os resultados apresentados correspondem à média dos tempos obtidos.

O tempo de execução foi medido utilizando a função `omp_get_wtime()`, que fornece uma medida de tempo com alta resolução adequada para avaliação de desempenho em aplicações paralelas.

Durante os experimentos, buscou-se manter condições semelhantes de execução, evitando a execução de aplicações concorrentes que pudessem interferir nos resultados.

4 Resultados

4.1 Tempos de execução

Os experimentos foram realizados variando o número de threads utilizadas na execução paralela do método de Jacobi. Para cada configuração, foram realizadas três execuções, sendo considerado o tempo médio.

O speedup foi calculado em relação à execução sequencial (1 thread), conforme a Equação:

$$S(p) = \frac{T(1)}{T(p)}$$

A eficiência foi calculada como:

$$E(p) = \frac{S(p)}{p}$$

Abaixo a Tabela 1 de dados sobre tempo, speedup e eficiência contra o programa paralelo com 1 thread.

Num. Threads	Tempo (s)	Speedup (vs. NTh1)	Eficiência
1	21.2	-	-
2	12.6	1.68	0.84
4	8.53	2.48	0.62
8	8.33	2.54	0.31
12	8.12	2.61	0.21
16	6.64	3.19	0.19

Table 1: Tempo Paralelo contra Numero de Thread 01

Abaixo, os gráficos para cada medida contra o Numero de Threads.

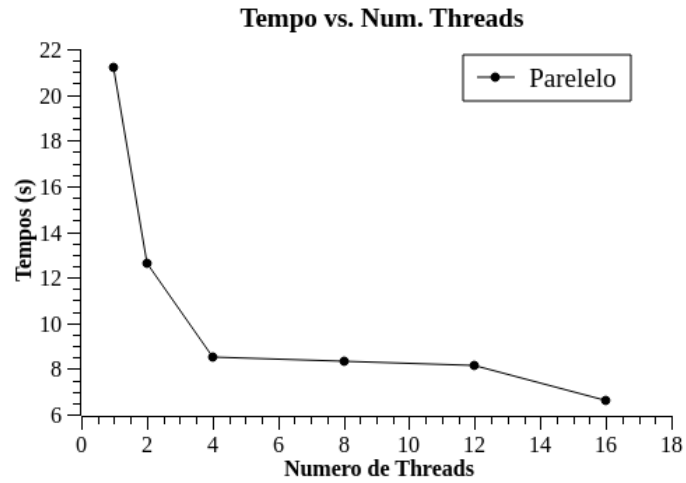


Figure 1: Tempo vs. Num. Threads

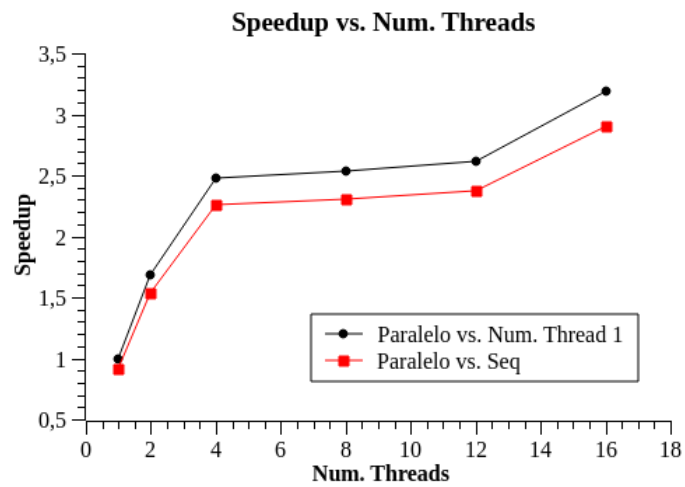


Figure 2: Speedup vs. Num. Threads

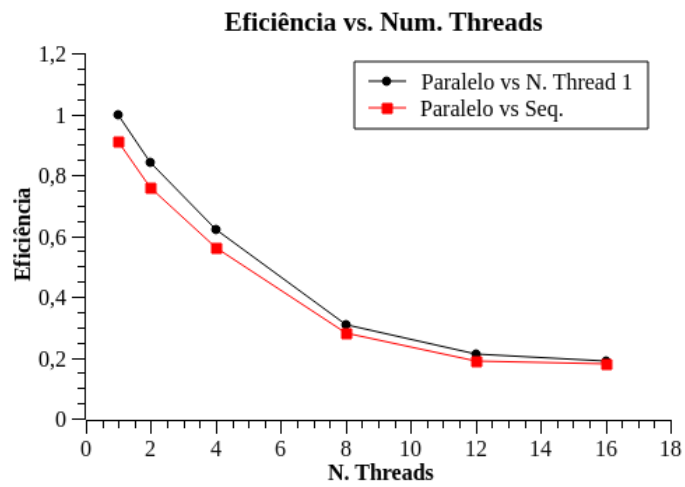


Figure 3: Eficiência vs. Num. Threads

Abaixa a Tabela 2 de dados sobre speedup e eficiência contra o programa sequencial.

Num. Threads	Speedup (vs. Seq.)	Eficiência
1	0.91	0.91
2	1.53	0.76
4	2.26	0.56
8	2.31	0.28
12	2.37	0.19
16	2.9	0.18

Table 2: Tempo Paralelo contra Sequencial Puro

Observa-se uma redução significativa no tempo de execução ao aumentar o número de threads, principalmente nas primeiras configurações.

Os resultados indicam que, embora o speedup aumente com o número de threads, a eficiência decai progressivamente, o que indica limitações de escalabilidade pela solução adotada ou limitação de ganho de desempenho sobre o número threads, seja pelo sistema operacional ou pelo hardware.

4.2 Convergência

A convergência do método de Jacobi depende diretamente das propriedades da matriz de coeficientes. Em particular, uma condição suficiente para a convergência é que a matriz seja diagonalmente dominante. No entanto, nem todos os sistemas utilizados nos experimentos satisfizeram essa propriedade.

Durante a execução dos testes, foram observados casos em que o método não convergiu. Nessas situações, o erro entre iterações não apresentou comportamento decrescente e, em alguns casos, cresceu progressivamente ao longo das iterações. Esse comportamento pode levar à divergência numérica, fazendo com que os valores do vetor solução assumam magnitudes muito elevadas, resultando em valores infinitos (`inf` ou `-inf`).

O critério de parada adotado foi baseado na norma Euclidiana da diferença entre iterações consecutivas:

$$\|X^{(k+1)} - X^{(k)}\|_2$$

Nos casos em que o método converge, esse erro tende a zero à medida que o número de iterações aumenta. Por outro lado, nos casos divergentes, o erro tende a crescer ou oscilar, impedindo que o critério de parada baseado em ϵ seja satisfeito.

Além disso, é importante destacar que a paralelização do algoritmo não influencia a convergência do método, pois não altera sua formulação matemática. A divergência observada está ligada exclusivamente às características do sistema linear e não à utilização de múltiplas threads.

Dessa forma, conclui-se que a garantia de convergência do método de Jacobi depende fortemente da construção da matriz de coeficientes, sendo fundamental propriedades como dominância diagonal para aplicações práticas.

É possível realizar alterações na forma qual a tolerância é disposta a depender do numero de threads. Entretanto, não realizado tal implementação nesse trabalho, mantendo a tolerância igual para todas threads.

Abaixo temos a uma Tabela 3 e Gráfico 4 iterações vs erro para a execução do algoritmo de dois sistemas de mesma ordem (4), numero de iterações (10) e $\epsilon = 0.1$ com um dos sistemas convergente e outro não convergente.

Iteração	Erro (Sistema 01)	Erro (Sistema 02)
1	15,53	24,57
2	12,51	28,5
3	11,44	33,68
4	10,02	39,6
5	8,81	46,58
6	7,82	54,84
7	6,89	64,52
8	6,1	75,93
9	5,38	89,34
10	4,76	105,14

Table 3: Erro dos sistemas para cada iteração

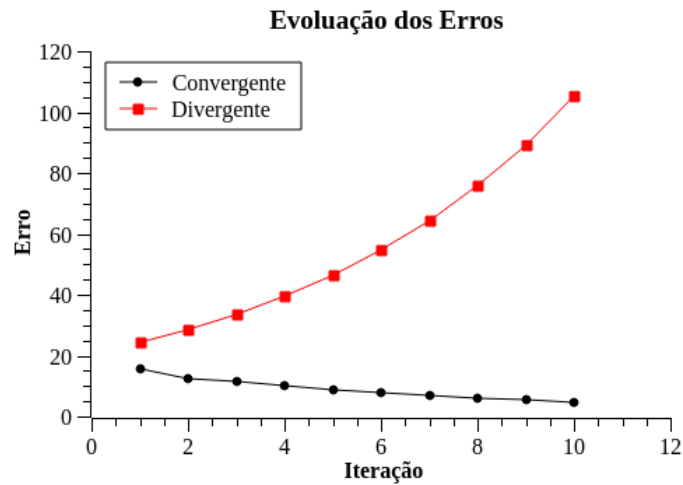


Figure 4: Evolução dos erros

4.3 Discussão

Os resultados demonstram que a paralelização do método de Jacobi utilizando OpenMP é eficaz na redução do tempo de execução, especialmente nas primeiras duplicações do número de threads.

Entretanto, observa-se que o ganho de desempenho não cresce linearmente com o número de threads. A partir de 4 threads, os ganhos tornaram-se menos expressivos, e em alguns casos há até aumento no tempo de execução, como observado na configuração com 8 threads.

Esse comportamento pode ser explicado por fatores como:

- Overhead de criação e sincronização de threads
- Limitações da arquitetura de hardware utilizada

Além disso, testes adicionais foram realizados visando otimizações, como a utilização de uma única região paralela e o uso de escalonamento dinâmico. No entanto, essas abordagens não apresentaram melhora de desempenho. A tentativa de reduzir o número de regiões paralelas resultou em piora do tempo total, possivelmente devido ao aumento da complexidade de sincronização. Da mesma forma, o uso de `schedule(dynamic)` não se mostrou vantajoso, uma vez que o custo computacional das iterações é uniforme, tornando o escalonamento estático mais adequado.

Ha de se ter em mente que tais experimentos foram executados em sistemas de pequeno porte, sistemas de grande porte (data center, servidores) poderiam ter ganho expressivo para maiores números de thread, tendo sua estagnação por overhead e hardware em um numero maior de threads do que aqui encontrado.

5 Conclusão

A implementação paralela do método de Jacobi utilizando OpenMP demonstrou ser uma abordagem eficiente para a resolução de sistemas lineares em termos de paralelização, proporcionando redução significativa no tempo de execução. Entretanto, em termos de solução de sistemas lineares, a limitação de quais sistemas convergirem e aplicáveis para o método reduz sua utilidade pratica.

Os resultados evidenciam que o paralelismo é mais efetivo em baixos níveis de concorrência, apresentando ganhos substanciais até um certo número de threads. A partir desse ponto, os benefícios diminuem devido a limitações inerentes ao paralelismo, como overhead e contenção de recursos.

Conclui-se que, embora a paralelização seja vantajosa, é fundamental considerar as características do problema e do ambiente de execução para obter o melhor desempenho possível.

6 Referencias

- [1] Vipin Kumar, Ananth Grama, Anshul Gupta, and George Karypis. *Introduction to parallel computing*, volume 110. Benjamin/Cummings Redwood City, CA, 1994.
- [2] Philip Wilkinson. *Parallel programming: Techniques and applications using networked workstations and parallel computers*, 2/E. Pearson Education India, 2006.